

Chaneg all the same words at the once

How to use:

- 1 Highlight the word you want to change (e.g., Channels)
- 2 Press Ctrl + Shift + L (or Cmd + Shift + L on Mac)
- 3 Every matching word in the file now has a cursor
- 4 Start typing the new name — all instances change simultaneously

This selects every occurrence, even inside other words. If you search for "Name", it will also match "Named" or "Namespace". Use Method 3 to avoid this.

Method 2: Incremental Selection (More Control)

This lets you select matches one by one, so you can skip certain instances.

Action	Windows/Linux	Mac
Select next match	Ctrl + D	Cmd + D
Undo last selection	Ctrl + U	Cmd + U

How to use:

- 1 Highlight the word you want to change
- 2 Press Ctrl + D (or Cmd + D) repeatedly to add more cursors
- 3 Each press selects the next occurrence
- 4 Made a mistake? Press Ctrl + U to undo the last selection
- 5 Type your new text — all selected spots update together

This is great when you only want to rename specific matches and leave others alone.

Method 3: Find and Replace (Safest for Exact Matches)

Use this when you want to replace whole words only and avoid changing partial matches.

Action	Windows/Linux	Mac
Open Find/Replace	Ctrl + H	Cmd + H

Steps:

- 1 Press Ctrl + H (or Cmd + H on Mac) to open the replace panel
- 2 Type the old text in the "Find" box
- 3 Type the new text in the "Replace" box
- 4 Click the "Match Whole Word" icon (Ab| word) — this prevents matching inside other words
- 5 Click "Replace All" to change everything at once

Comparison Table

Method	Best For	Speed	Precision
Ctrl + Shift + L	Renaming all matches quickly	Fastest	Low (matches substrings)
Ctrl + D	Selective renaming	Fast	High (you control each)
Ctrl + H + whole word	Safe, precise renaming	Medium	Highest

This works to unblock browser if task kill does not

```
mkdir C:\Dev\Websites
```

```
Copy-Item -Recurse -Force "C:\Users\matth\OneDrive\Desktop\Websites\Part2" "C:\Dev\Websites\Part2"
```

```
cd C:\Dev\Websites\Part2
```

```
dotnet clean
```

```
dotnet run
```

This Kills all instances

```
Get-Process dotnet -ErrorAction SilentlyContinue | Stop-Process -Force
```

This makes the db

```
sqlite3 Users.db -init Users.sql
```

```
sqlite3 Employees.db --init Employees.sql
```

sqlite3 Websites.db --init Websites.sql

Part 1 (Console + SQLite + EF Core) — “where is each bit in the tutorial PDFs?”

This README maps **every part** of your Part 1 app to the **Week 1–5 tutorial PDFs** you linked, and shows a **small code snippet** for each bit (with comments so you know what to do).

Important: the PDFs mention “Video X” a lot, but you said you’re **not watching videos**.

So in this README I’m pointing you to what you can actually see in the PDFs:

- **Which Week PDF**
- **Which page** (the PDFs show “Page X of Y”)
- **The heading text** next to it

1) Create / build / run a console app (project basics)

- **Where in PDFs**: **Week 1 – Getting Started.pdf**
 - **Page 1 of 7**: “Create a Console Application” (shows ``dotnet new console``, ``dotnet build``, ``dotnet run``)
 - **Page 2 of 7**: “Executing the console application” (running from ``bin/Debug/net8.0``)

Snippet (create/build/run):

````powershell`

# Create a new console project (like Part1.csproj)

**dotnet new console**

**# Build it (creates bin/ + obj/)**

**dotnet build**

**# Run it (executes Program.cs)**

**dotnet run**

...

---

**## 2) Output text (print your name + student number)**

- **\*\*Where in PDFs\*\*: \*\*Week 1 – Getting Started.pdf\*\***

- **\*\*Page 1 of 7\*\*: “Create a Console Application” (shows the initial `Program.cs` + `Console.WriteLine`)**

**\*\*Snippet (Console.WriteLine):\*\***

**```csharp**

**// Print fixed text to the console (your required banner lines)**

**Console.WriteLine("Your Name");**

**Console.WriteLine("Your Student/Banner Number");**

...

---

### **## 3) Get user input (ask for a site name)**

- **\*\*Where in PDFs\*\*:**

- **\*\*Week 1 – Getting Started.pdf\*\***

- **\*\*Page 4 of 7\*\*:** “Inputting a String value” (shows `Console.ReadLine()` style input)

- **\*\*Week 2 – Programming Fundamentals.pdf\*\***

- **\*\*Page 3 of 18\*\*:** “Inputting Strings” (same idea, more practice examples)

**\*\*Snippet (prompt + ReadLine):\*\***

```
```csharp
```

```
// Ask the user for input
```

```
Console.Write("Enter a site name: ");
```

```
// Read what they typed (ReadLine can be null, so we guard it)
```

```
var siteName = Console.ReadLine()?.Trim() ?? string.Empty;
```

```
```
```

```

```

### **## 4) Keep the console window open (Press any key)**

- **\*\*Where in PDFs\*\*:** **\*\*Week 1 – Getting Started.pdf\*\***

- **\*\*Page 2 of 7\*\*:** “Executing the console application” (adds `ReadKey` so it doesn’t close instantly)

**\*\*Snippet (ReadKey pause):\*\***

```
```csharp
```

```
// Pause so the window stays open when run by double-clicking the .exe
```

```
Console.WriteLine();
```

```
Console.WriteLine("Press any key to exit.");
```

```
Console.ReadKey(intercept: true);
```

```
```
```

```

```

**## 5) Create the SQLite database from a `.sql` script**

Your coursework includes `StreamingSites.sql` (CREATE TABLE + INSERTs) and a built `StreamingSites.db`.

- **\*\*Where in PDFs\*\*:** **\*\*Week 3 – Connecting to a Database.pdf\*\***

- **\*\*Page 6 of 13\*\*:** “Working with SQL” (create the `.sql` script)

- **\*\*Page 6–7 of 13\*\*:** “Executing SQL / Adding Multiple Records” (run `sqlite3 ... -init ...` to generate the `.db`)

**\*\*Snippet (make `.db` from `.sql`):\*\***

```
```powershell
```

```
# Create (or recreate) the SQLite database file from the SQL script
```

```
sqlite3 StreamingSites.db -init StreamingSites.sql
```

If you end up inside the sqlite prompt, exit like this:

.exit

...

6) Add EF Core packages to the project (.csproj)

This is what makes DbContext, DbSet<>, and UseSqlite(...) work.

- **Where in PDFs**: **Week 3 – Connecting to a Database.pdf**

- **Page 7 of 13**: “Referencing Entity Framework Core” (adds the package reference in the .csproj)

Snippet (.csproj PackageReference):

```xml

<!-- Add these packages so C# can talk to SQLite using EF Core -->

<ItemGroup>

<PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.11" />

<PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="8.0.11" />

</ItemGroup>

...

---

## ## 7) Create an entity class that matches the table (your `Channel`)

An “entity” class represents **one row** in your table.

- **Where in PDFs:** **Week 3 – Connecting to a Database.pdf**

- **Page 7 of 13:** “Creating an Entity Class” (class properties match the table fields)

**Snippet (simple entity shape):**

```
```csharp
// This class should match your database table columns (names + types)

public class Channel
{
    public int ID { get; set; }    // matches ID column

    public string Name { get; set; } = ""; // matches Name column

    public string Url { get; set; } = ""; // matches URL column
}
```
```

In your real code you also used `[Table]` / `[Column]` attributes — that’s just making the mapping explicit.

---

## ## 8) Create a DbContext class (your `StreamingSitesDatabase`)

The “context” is the thing that **\*\*connects\*\*** your code to the database file.

- **\*\*Where in PDFs\*\***: **\*\*Week 3 – Connecting to a Database.pdf\*\***

- **\*\*Page 8 of 13\*\***: “Creating a Context Class” + “Inheriting the DbContext Class”

**\*\*Snippet (DbContext):\*\***

```
```csharp
```

```
// This is the database “connection + tables” in C#
```

```
public class StreamingSitesDatabase : DbContext
```

```
{
```

```
    public StreamingSitesDatabase(DbContextOptions<StreamingSitesDatabase> options) :  
base(options) { }
```

```
    // This represents the Channels table as a queryable set of Channel rows
```

```
    public DbSet<Channel> Channels { get; set; } = null!;
```

```
}
```

```
```
```

```

```

**## 9) Define a `DbSet<>` for the table you want (your `Channels`)**

- **\*\*Where in PDFs\*\***: **\*\*Week 3 – Connecting to a Database.pdf\*\***

- **\*\*Page 8 of 13\*\***: “Defining a DbSet Property”

**\*\*Snippet (DbSet):\*\***

```
```csharp
```

```
// DbSet<Channel> = “table of Channels”
```

```
public DbSet<Channel> Channels { get; set; } = null!;
```

```
...
```

```
---
```

10) Connect to the `.db` file using `UseSqlite(...)`

- ****Where in PDFs**:** ****Week 3 – Connecting to a Database.pdf****

- ****Page 9 of 13**:** “Connecting to the Database” (shows the standard “options builder” code)

****Snippet (options + context):****

```
```csharp
```

```
// Build EF Core options that point at your database file
```

```
var options = new DbContextOptionsBuilder<StreamingSitesDatabase>()
```

```
.UseSqlite("Data Source=StreamingSites.db") // path to the SQLite database file
```

```
.Options;
```

```
// Create the context (this opens the connection when EF needs it)
```

```
using var db = new StreamingSitesDatabase(options);
```

...

---

**## 11) IMPORTANT: database file must be beside the executable (bin folder)**

This is the classic “works in VS Code but fails when running the .exe” problem.

- **\*\*Where in PDFs\*\*:** **\*\*Week 3 – Connecting to a Database.pdf\*\***

- **\*\*Page 13 of 13\*\*:** “Running a Database Application”

- It literally says you must have the `.db`` beside the executable in `bin/Debug/net8.0``.

**\*\*What your project does\*\*:**

- Your `Part1.csproj`` already copies `StreamingSites.db`` to the output folder.

**\*\*Snippet (copy on build):\*\***

```xml`

`<!-- Copies StreamingSites.db into bin/Debug/net8.0 automatically -->`

`<ItemGroup>`

`<None Update="StreamingSites.db">`

`<CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>`

`</None>`

`</ItemGroup>`

...

12) LINQ filtering (find the row that matches what the user typed)

Your app searches the `Channels` table for the channel name.

- ****Where in PDFs**:**

- ****Week 4 – Working with Linq (1).pdf****

- ****Page 8 of 9**:** “Filtering Information” (shows filtering with `Where(...)`)

- ****Week 3 – Connecting to a Database.pdf****

- ****Page 11 of 13**:** “Selecting a Record” (the idea of “pick one record once you’ve filtered”)

****Snippet (Where + FirstOrDefault):****

```
```csharp
```

```
// Query the Channels table and pick the first match (or null if none)
```

```
var match = db.Channels
```

```
 .Where(c => c.Name == siteName)
```

```
 .FirstOrDefault();
```

```
```
```

****Why `FirstOrDefault()` matters:****

- It returns `null` when nothing matches, so you can print a “not found” message.

13) Case-insensitive matching (user can type netflix / Netflix / NETFLIX)

- ****Where in PDFs****: this is an ****extension**** of the LINQ filtering from ****Week 4 – Working with Linq (1).pdf****

- ****Page 8 of 9****: “Filtering Information” shows ``Where(...)``. Case-insensitive comparison is a normal real-world improvement on top of that.

****Snippet (case-insensitive):****

```
```csharp
```

```
// Compare both values in the same case to ignore case differences
```

```
var match = db.Channels
```

```
 .Where(c => c.Name.ToLower() == siteName.ToLower())
```

```
 .FirstOrDefault();
```

```
...
```

```

```

## **## 14) Output the result (URL or “not found”)**

- **\*\*Where in PDFs\*\***:

- **\*\*Week 1 – Getting Started.pdf\*\***: basic output with ``Console.WriteLine``

- **\*\*Week 2 – Programming Fundamentals.pdf\*\***: lots of examples using ``$"..."`` string interpolation (used throughout)

**\*\*Snippet (if/else output):\*\***

```
```csharp
// Decide what to print based on whether a match was found
if (match is null)
{
    Console.WriteLine($"No URL found for '{siteName}'");
}
else
{
    Console.WriteLine($"The URL for {match.Name} is {match.Url}");
}
```
```

---

## **## 15) What Week 5 (Razor) is for**

- **\*\*Where in PDFs\*\*:** **\*\*Week 5 – Using Razor.pdf\*\*** (all of it)
- **\*\*How it relates to Part 1\*\*:** it doesn't, unless the test asked for a **\*\*web\*\*** version.

**Week 5 shows how to:**

- create a web app (`dotnet new web`)
- serve Razor pages
- connect Razor to SQLite (re-uses Week 3/4 concepts)

---

## **## Quick “this app is basically...”**

- **\*\*Week 1\*\***: console app + input/output + “press any key”
- **\*\*Week 3\*\***: SQLite ``.sql` → `.db``, EF Core packages, entity class, context class, connecting to DB, and the “DB must be in bin folder” rule
- **\*\*Week 4\*\***: LINQ filtering to find the right record